The Journal of Open Source Software

Stellarmesh: A Meshing Library for Fusion Neutronics Workflows

Alexander Koen [1], **Paul K. Romano** [1,2], **and Samuel Pasmann** [1]

1 Thea Energy, Inc., Kearny, NJ, USA **2** Argonne National Laboratory, Lemont, IL, USA

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 06 November 2026
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

Summary

Neutron–material interactions are a central area of study in nuclear science and engineering. As neutrons interact with the surrounding materials, they undergo a number of nuclear reactions. These interactions are often studied using Monte Carlo simulation methods. Stellarmesh is an MIT-licensed Python library that preprocesses, meshes, and converts computer-aided design (CAD) geometries into a format suitable for analysis with the open-source particle transport code OpenMC (Romano et al., 2015) and the Direct Accelerated Geometry Monte Carlo (DAGMC) CAD-transport extension (Wilson et al., 2010).

Statement of Need

The development of Stellarmesh is motivated by the increase in demand for an open-source neutronics software ecosystem, the need to perform analysis on complex CAD geometries, and the unique mesh requirements imposed by the DAGMC mesh transport library.

Open-Source Neutronics

In recent years, open-source tools for neutronics have gained traction within academia and private industry (Peterson et al., 2024). This interest is due in part to export-control restrictions (Los Alamos National Laboratory, 2026) that limit access to traditional transport codes such as MCNP (Kulesza et al., 2022), and in part to the capability of the open-source tools themselves. These tools—which include the Monte Carlo transport code OpenMC (Romano et al., 2015), the mesh-based particle tracking library DAGMC (Wilson et al., 2010), the parametric CAD libraries CadQuery (CadQuery contributors, 2025a) and build123d (Maitland & others, 2025), and the mesh generator Gmsh (Geuzaine & Remacle, 2009)—form a growing ecosystem of open-source libraries that compose flexibly in standard Python environments.

CAD Geometry

Monte Carlo neutron transport codes traditionally use constructive solid geometry (CSG) to represent experiment geometry (Romano et al., 2015; Shriwise et al., 2022). This is suitable for simple geometries. However, most complex models are developed using boundary-representation CAD software. These models can then be discretized into surface or volume meshes using a variety of meshing codes. While approaches such as GEOUNED (Catalán et al., 2024) exist to translate boundary-representation models to CSG, the spline surfaces common in these models must ultimately be discretized in either case (Catalán et al., 2024; Häußler et al., 2017).

DAGMC (Wilson et al., 2010) is a library that enables ray-traced queries on triangular surface meshes derived from CAD geometry, and that can be used in conjunction with OpenMC to support transport on such meshes (OpenMC developers, 2026).

DRAFT

DAGMC Mesh Requirements

DAGMC needs its meshes to be stored in the HDF5-based H5M format of the Mesh-Oriented datABase (MOAB) (MOAB developers, 2026; Tautges et al., 2004). Each mesh comprises a set of closed logical volumes bounded by surface meshes, for which Figure 1 shows a visual representation. These meshes must adhere to the following requirements (Smith et al., 2010; The UW CNERG Team, 2022a):

- **Orientability:** The normals of every triangle on a surface must point toward the same volume.
- **Shared-surface topology:** Where adjoining volumes touch, they must share a single surface at the point of intersection. Each surface must label which logical volume the triangle normals point into and which they point out of.
- **Non-intersection:** Surface meshes must not intersect themselves or other meshes.
- **Watertightness:** While meshes with at least one shared surface are inherently non-manifold, the set of surfaces bounding a volume must not contain holes or open edges.

DAGMC does not mesh geometries itself. As a result, a separate library is required to convert CAD geometries to DAGMC meshes with correct metadata.

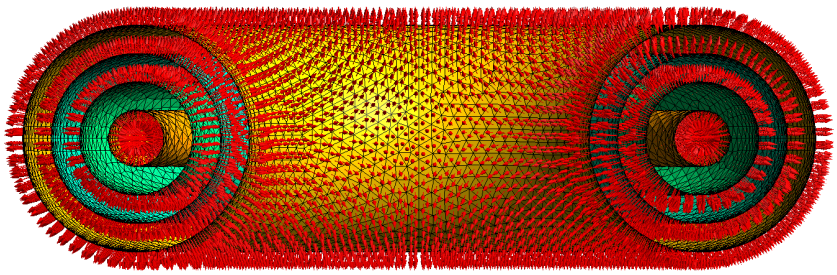


Figure 1: Stellarmesh render of a concentric torus model. Each inner surface is shared by the solid tori it bounds and tagged according to the direction of its normals.

State of the Field

DAGMC was first released as a MOAB tool in 2007 (MOAB developers, 2007). At that time, it supported only geometries from the proprietary ACIS geometry kernel, and these models were meshed at runtime using the bundled ACIS faceting routines (MOAB developers, 2007; The UW CNERG Team, 2010). In 2010, faceting became a preprocessing step (Jackson, 2010) and watertight-sealing methods were published (Smith et al., 2010).

As of 2014, a typical DAGMC workflow relied on the proprietary meshing library Cubit (Coreform LLC, 2020, 2026a) and resembled the following (The UW CNERG Team, 2014):

1. Import, imprint, and merge geometry using Cubit
2. Assign volumes to material groups in Cubit
3. Export the Cubit model in ACIS format
4. Facet it with `dagmc_preproc`

5. Seal the result with `make_watertight`

In 2016, the developers of DAGMC added an `export_dagmc` command to the Svalinn Cubit plugin, enabling users to perform steps 3 through 5 from within Cubit (The UW CNERG Team, 2022b)[1].

In 2018, the first fully open-source alternative workflow was enabled with the development of `occ_faceter` (Davis & Mason, 2018), a C++ library that meshed geometries from the open-source OpenCASCADE (OCC) geometry kernel and converted them to DAGMC format. However, no packaged release of the project was ever made.

The first packaged Python DAGMC export came as part of the 0.0.12 release of Paramak—an OCC-based tokamak modeling library—in September 2020 (*Paramak 0.0.12*, 2020; Shimwell et al., 2021). Paramak exported its own reactor objects as H5M either through Cubit or through direct STL export and conversion. These two export methods were later extracted from Paramak in 2021 and repackaged as `cad_to_h5m` (Shimwell et al., 2022) and `stl_to_h5m` (Shimwell & Shriwise, 2022) respectively. However, `cad_to_h5m` still had a dependency on Cubit and `stl_to_h5m` did not support shared-surface topology.

In 2022, two further libraries, `brep_to_h5m` (Shimwell, 2022a) and `vertices_to_h5m` (Shimwell, 2022b), were released. `brep_to_h5m` meshed OpenCASCADE BREP files with Gmsh and exported them to H5M with `stl_to_h5m` and later with `vertices_to_h5m`, which allowed the creation of H5M files from vertices stored in memory.

The first general-purpose, open-source converters came later in 2022 with the release of CAD-to-OpenMC (Knudsen & Chierici, 2025) and CAD-to-DAGMC (Shimwell, 2026a). These libraries were the first to take arbitrary CAD geometries from OpenCASCADE, imprint and merge them, and convert them to tagged H5M models. However, until late 2023, neither tool supported shared-surface topology (Knudsen, 2023; Shimwell, 2023).

In September 2023, the authors released the first version of Stellarmesh. Its development was motivated by the fragmentation of existing libraries and the lack of any self-contained converter library that wrote all required DAGMC metadata. Stellarmesh aims to be feature-complete, stable, documented, and correct.

Since then, Cubit, CAD-to-OpenMC, CAD-to-DAGMC, and Stellarmesh have each matured with active maintenance and new feature releases. Thus, at this time, there are four actively maintained libraries for DAGMC mesh preparation. Table 1 compares them on a subset of desired attributes including license, geometry input support, meshing backends, and testing methodology.

Table 1: Comparison of Stellarmesh, CAD-to-DAGMC (Shimwell, 2026a), CAD-to-OpenMC (Knudsen & Chierici, 2025), and Coreform Cubit (Coreform LLC, 2026a) on a subset of features.

Feature	Stellarmesh	CAD-to-DAGMC	CAD-to-OpenMC	Coreform Cubit
License	MIT	MIT [2]	MIT	Proprietary; export-controlled (EAR99) (Coreform LLC, 2024)

[1] In 2023, Coreform added native DAGMC export to Cubit (Coreform LLC, 2023), and the Svalinn plugin has been unsupported since June 2025 (The UW CNERG Team, 2022b).

DRAFT

Feature	Stellarmesh	CAD-to-DAGMC	CAD-to-OpenMC	Coreform Cubit
Input geometry formats	CadQuery or build123d objects, OCC pointers, STEP, OpenCASCADE BREP	CadQuery objects, STEP	STEP	ACIS, Pro/E, CATIA (native); STEP, IGES (translated) (Coreform LLC, 2026c)
Surface meshing backends	OCC and Gmsh	Gmsh and `cad-to-dagmc-mesher`	OCC and Gmsh	Cubit internal
Volume meshing backends	Gmsh	Gmsh and `cad-to-dagmc-mesher`	None	Cubit internal
Python environment requirements	ABI-compatible MOAB and Gmsh	CadQuery	CadQuery; MOAB	Installed Cubit application + license
Surface boundary conditions	Yes	No	No	Yes
Release cadence	12 releases, 2023-09 to 2026-02	58 releases, 2022-08 to 2026-10; no stable release	26 releases, 2022-07 to 2025-02	About 3 per year (Coreform LLC, 2026b)
Testing	CAD-vs-CSG benchmarks	CAD-vs-CSG benchmarks from `model_benchmark_zoo` (Shimwell, 2026c)	CAD-vs-CSG benchmarks from `model_benchmark_zoo` (Shimwell, 2026c)	Undetermined

Software Design

Geometry Kernel

Boundary-representation geometry operations are performed by the OCC (CadQuery contributors, 2025b; Open Cascade SAS, 2024) geometry kernel. OCC provides STEP file import (Open Cascade SAS, 2024) and is the geometry kernel underlying CadQuery (CadQuery contributors, 2025a) and build123d (Maitland & others, 2025), so Stellarmesh can process their objects directly. Stellarmesh additionally exposes OCC's imprint-and-merge functionality (Open Cascade SAS, 2024) to create shared-surface topology.

Meshing Backends

Stellarmesh provides two meshing backends. For each, a subset of useful options is exposed for mesh control.

The OCC backend uses OpenCASCADE's built-in meshing functionality to produce surface meshes. This backend supports linear and angular deviation tolerances (Open Cascade SAS, 2024), which can be used to ensure that volumes separated by a known distance do not overlap.

The Gmsh (Geuzaine & Remacle, 2009) backend does not support these tolerances, but in contrast to the OCC backend—whose meshes are primarily tailored to visualization—its

[2] At the time of writing, the source code for the most recent version of `cad-to-dagmc-mesher` (0.3.7) is not public (Shimwell, 2026b). Its public source stops at version 0.3.2.

DRAFT

algorithms are optimized to produce high-quality triangle elements with lower aspect ratios (Geuzaine & Remacle, 2026). In addition, Gmsh can generate tetrahedral volume meshes that Stellarmesh converts to MOAB format (Tautges et al., 2004) for unstructured volume-mesh tallies in OpenMC (Romano et al., 2015).

Stable File Format

Regardless of the meshing backend, all Stellarmesh meshes are stored in Gmsh MSH 4.1 format (Geuzaine & Remacle, 2026), with metadata encoded in Gmsh physical group names as URL-style `key=value` strings.

Because this metadata format is stable and documented, anyone can create a custom mesh backend for Stellarmesh. For example, Basalt (Koen, 2026) is a mesh backend for Stellarmesh developed by the authors that meshes proprietary Parasolid (Siemens Digital Industries Software, 2026) geometries from commercial CAD software. Stellarmesh then provides the DAGMC file creation and metadata tagging, allowing for simple substitution of geometry sources.

Testing and Verification

The Stellarmesh repository runs continuous integration tests on each commit. These tests perform Monte Carlo transport on a selection of CSG and Stellarmesh models of the same geometries to assert that the results match to within a fixed relative tolerance of 2.5%.

Workflow

Figure 2 shows a symbolic representation of a typical end-to-end neutronics workflow using Stellarmesh. In this example, a stellarator equilibrium model is loaded using DESC (Conlin et al., 2023; Dudt et al., 2023; Dudt & Kolemen, 2020; Panici et al., 2023). From the DESC equilibrium, the blanket geometry is modeled with build123d. The geometry is imported into Stellarmesh, and a DAGMC surface mesh is generated. This surface mesh is then used for transport in OpenMC.

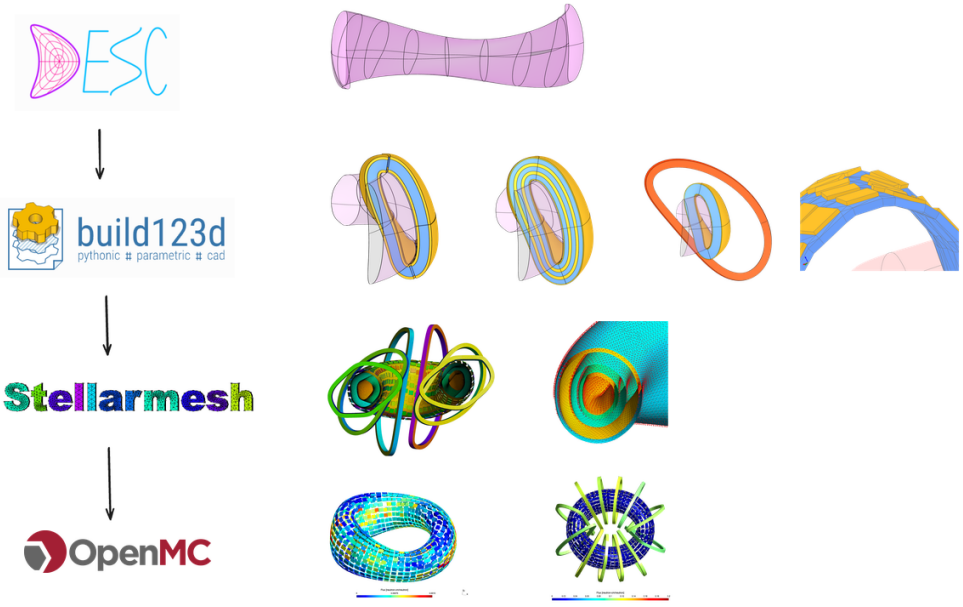


Figure 2: An example neutronics workflow. A stellarator equilibrium is loaded using DESC (Dudt & Kolemen, 2020). The blanket CAD geometry is built from this equilibrium using build123d. The DAGMC surface mesh is generated using Stellarmesh. Transport is performed using OpenMC.

DRAFT

Research Impact Statement

Stellarmesh models have been used in five peer-reviewed journal publications. All CAD neutronics models for Thea Energy's initial conceptual publication in *Nuclear Fusion* (Gates et al., 2025) and for the Helios Stellarator Design special issue of *Fusion Engineering and Design* (Pasmann et al., 2026; Swanson et al., 2026) were developed with Stellarmesh. Figure 3 shows an OpenMC heating tally performed on a Stellarmesh model.

Stellarmesh has additionally been used in the development of material volume fraction estimation (Romano et al., 2025) and shutdown dose rate analysis (Romano et al., 2027) in OpenMC.

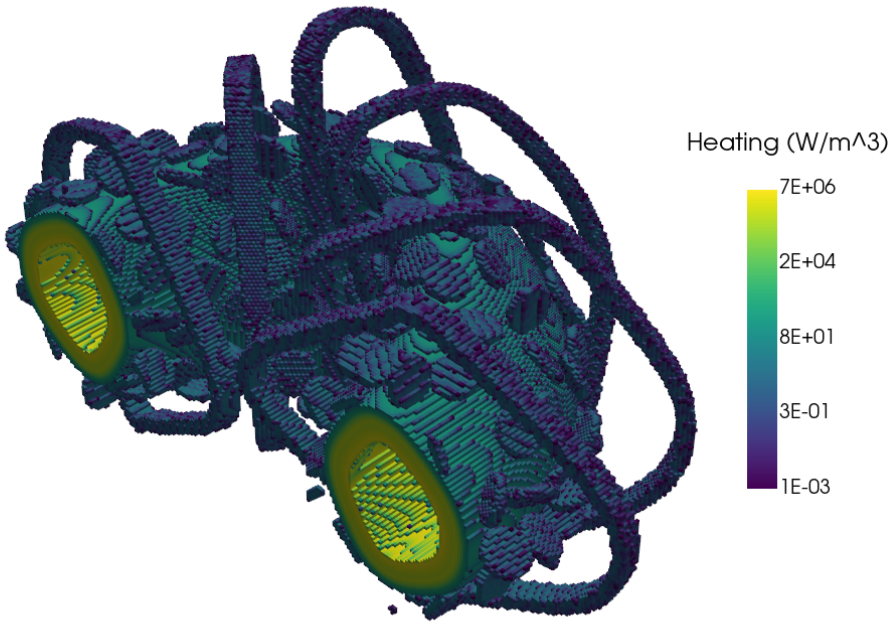


Figure 3: OpenMC heating tally performed on a Stellarmesh model of Thea Energy's Helios conceptual design, reproduced from Fig. 15 of Pasmann et al. (2026) (CC BY 4.0). The tally is scored and displayed on a voxel grid, while the Stellarmesh surface mesh used for transport conforms to the geometric surfaces.

AI Usage Disclosure

Stellarmesh versions through v1.0.0 (September 2025) were designed and written without generative-AI assistance. After v1.0.0, the developers have used Google and Anthropic models for code assistance and documentation. All model output has been verified.

Acknowledgments

Development of Stellarmesh is funded by Thea Energy. Koen, Pasmann, and Romano are employed by Thea Energy. The authors would like to thank Jonathan Shimwell, Erik B. Knudsen, and Ahnaf Tahmid Chowdhury for providing feedback and historical context.

References

CadQuery contributors. (2025a). *CadQuery* (Version 2.6.1). https://doi.org/10.5281/zenodo.14590990

CadQuery contributors. (2025b). *OCP: Python wrapper for OCCT* (Version 7.8.1.2). https://github.com/CadQuery/OCP

Catalán, J. P., Sauvan, P., García, J., Alguacil, J., Ogando, F., & Sanz, J. (2024). GEOUNED: A new conversion tool from CAD to Monte Carlo geometry. *Nuclear Engineering and Technology*, *56*(6), 2404–2411. https://doi.org/10.1016/j.net.2024.01.052

Conlin, R., Dudt, D. W., Panici, D., & Kolemen, E. (2023). The DESC stellarator code suite. Part 2. Perturbation and continuation methods. *Journal of Plasma Physics*, *89*(3), 955890305. https://doi.org/10.1017/S0022377823000399

Coreform LLC. (2020). *Trelis is now Coreform Cubit*. https://coreform.com/news/trelis-is-now-coreform-cubit/

Coreform LLC. (2023). *Release notes for Coreform Cubit 2023.11*. https://coreform.com/coreform-cubit-old/release-notes/v2023-11/

Coreform LLC. (2024). *Coreform export policy*. https://coreform.com/export/

Coreform LLC. (2026a). *Coreform Cubit* (Version 2026.8). https://coreform.com/coreform-cubit/

Coreform LLC. (2026b). *Coreform Cubit release notes*. https://coreform.com/coreform-cubit-old/release-notes/

Coreform LLC. (2026c). *Defining the geometric model, Coreform Cubit help v2026.8*. https://coreform.com/cubit_help/latest/item/importing.htm

Davis, A., & Mason, S. (2018). *OCC faceter: Tools for the production, modification and serialisation of geometries for DAGMC*. https://github.com/makeclean/occ_faceter

Dudt, D. W., Conlin, R., Panici, D., & Kolemen, E. (2023). The DESC stellarator code suite Part 3: Quasi-symmetry optimization. *Journal of Plasma Physics*, *89*(2), 955890201. https://doi.org/10.1017/S0022377823000235

Dudt, D. W., & Kolemen, E. (2020). DESC: A stellarator equilibrium solver. *Physics of Plasmas*, *27*(10), 102513. https://doi.org/10.1063/5.0020743

Gates, D. A., Aslam, S., Berzin, B., Bonofiglo, P., Cote, A., Dudt, D. W., Flom, E., Fort, D., Koen, A., Kruger, T. G., Kumar, S. T. A., Martin, M. F., Ottaviano, A., Pasmann, S., Romano, P. K., Swanson, C. P. S., Tang, L., Winkler, E., & Wu, R. (2025). Stellarator fusion systems enabled by arrays of planar coils. *Nuclear Fusion*, *65*(2), 026052. https://doi.org/10.1088/1741-4326/ada56c

Geuzaine, C., & Remacle, J.-F. (2009). Gmsh: A 3-D finite element mesh generator with built-in pre- and post-processing facilities. *International Journal for Numerical Methods in Engineering*, *79*(11), 1309–1331. https://doi.org/10.1002/nme.2579

Geuzaine, C., & Remacle, J.-F. (2026). *Gmsh reference manual* (4.15.2 ed.). https://gmsh.info/doc/texinfo/gmsh.html

Häußler, A., Fischer, U., & Warmer, F. (2017). Verification of different Monte Carlo approaches for the neutronic analysis of a stellarator. *Fusion Engineering and Design*, *124*, 1207–1210. https://doi.org/10.1016/j.fusengdes.2017.04.010

Jackson, S. (2010). *Add dagmc_preproc tool within dagmc dir; add command line option parser util*. MOAB commit 7fc6477e4ea8b2f94b8e5bd807883c8281072410. https://bitbucket.org/fathomteam/moab/commits/7fc6477e4ea8b2f94b8e5bd807883c8281072410

DRAFT

Knudsen, E. B. (2023). *CAD_to_OpenMC* (Version 0.2.6). https://pypi.org/project/CAD-to-OpenMC/0.2.6/

Knudsen, E. B., & Chierici, L. (2025). CAD_to_OpenMC: From CAD design to particle transport. *Journal of Open Source Software*, *10*(110), 7710. https://doi.org/10.21105/joss.07710

Koen, A. (2026). *Basalt: A nuclear meshing library for NX and Parasolid based on the Simmetrix mesh suite* (Version 0.2). https://github.com/Thea-Energy/basalt

Kulesza, J. A., Adams, T. R., Armstrong, J. C., Bolding, S. R., Brown, F. B., Bull, J. S., Burke, T. P., Clark, A. R., Forster, R. A., III, Giron, J. F., Grieve, T. S., Josey, C. J., Martz, R. L., McKinney, G. W., Pearson, E. J., Rising, M. E., Solomon, C. J., Jr., Swaminarayan, S., Trahan, T. J., … Zukaitis, A. J. (2022). *MCNP code version 6.3.0 theory & user manual* (LA-UR-22-30006, Rev. 1). Los Alamos National Laboratory. https://doi.org/10.2172/1889957

Los Alamos National Laboratory. (2026). *How to get the MCNP code*. https://mcnp.lanl.gov/how_to_get_the_mcnp_code.html

Maitland, R., & others. (2025). *build123d: A Python-based parametric CAD library* (Version 0.9.1). https://doi.org/10.5281/zenodo.14872323

MOAB developers. (2007). *MOAB: A mesh-oriented database* (Version 3.0.0). https://bitbucket.org/fathomteam/moab/commits/tag/3.0.0

MOAB developers. (2026). *MOAB README*. MOAB repository, commit a5ae4bc75dcac0a6bdd87454a3b0 https://bitbucket.org/fathomteam/moab/src/a5ae4bc75dcac0a6bdd87454a3b049e7b3c06eed/README.md

Open Cascade SAS. (2024). *Open CASCADE Technology (OCCT)* (Version 7.8.1). https://github.com/Open-Cascade-SAS/OCCT

OpenMC developers. (2026). *Using CAD-based geometry, OpenMC documentation*. https://docs.openmc.org/en/stable/usersguide/geometry.html#using-cad-based-geometry

Panici, D., Conlin, R., Dudt, D. W., Unalmis, K., & Kolemen, E. (2023). The DESC stellarator code suite. Part 1. Quick and accurate equilibria computations. *Journal of Plasma Physics*, *89*(3), 955890303. https://doi.org/10.1017/S0022377823000272

Paramak 0.0.12. (2020). Python Package Index. https://pypi.org/project/paramak/0.0.12/

Pasmann, S., Riel, A. van, Khera, U., Romano, P. K., Koen, A., Donovan, M., Swanson, C. P. S., & Gates, D. A. (2026). Preliminary nuclear and thermal analysis of a dual-coolant lead-lithium blanket concept for the Helios fusion power plant. *Fusion Engineering and Design*, *231*, 115951. https://doi.org/10.1016/j.fusengdes.2026.115951

Peterson, E. E., Romano, P. K., Shriwise, P. C., & Myers, P. A. (2024). Development and validation of fully open-source R2S shutdown dose rate capabilities in OpenMC. *Nuclear Fusion*, *64*(5), 056011. https://doi.org/10.1088/1741-4326/ad32dd

Romano, P. K., Horelik, N. E., Herman, B. R., Nelson, A. G., Forget, B., & Smith, K. (2015). OpenMC: A state-of-the-art Monte Carlo code for research and development. *Annals of Nuclear Energy*, *82*, 90–97. https://doi.org/10.1016/j.anucene.2014.07.048

Romano, P. K., Pasmann, S., Shriwise, P. C., & Swanson, C. P. S. (2025). Computing material volume fractions on a superimposed mesh as applied to Monte Carlo particle transport simulations. *Fusion Engineering and Design*, *220*, 115364. https://doi.org/10.1016/j.fusengdes.2025.115364

Romano, P. K., Peterson, E. E., Shriwise, P. C., Pasmann, S. L., & Swanson, C. P. S. (2027). Integrated sub-element mesh-based R2S shutdown dose rate calculations in OpenMC. *Fusion Engineering and Design*, *234*, 116095. https://doi.org/10.1016/j.fusengdes.2026.116095

DRAFT

Shimwell, J. (2022a). *BRep to h5m: Converts Brep CAD geometry files to h5m geometry files compatible with DAGMC simulations* (Version 0.4.2). https://github.com/fusion-energy/brep_to_h5m

Shimwell, J. (2022b). *Vertices to h5m: Converts mesh vertices and connectivity to h5m geometry files compatible with DAGMC simulations* (Version 0.1.7). https://github.com/fusion-energy/vertices_to_h5m

Shimwell, J. (2023). *CAD to DAGMC* (Version 0.5.0). https://github.com/fusion-energy/cad_to_dagmc/releases/tag/0.5.0

Shimwell, J. (2026a). *CAD to DAGMC: Convert CAD geometry or CadQuery assemblies to DAGMC h5m files* (Version 0.15.1). https://github.com/fusion-energy/cad_to_dagmc

Shimwell, J. (2026b). *cad-to-dagmc-mesher: CDT surface mesher with DAG scheduler for DAGMC geometry* (Version 0.3.7). https://pypi.org/project/cad-to-dagmc-mesher/

Shimwell, J. (2026c). *Model Benchmark Zoo: A collection of CAD and equivalent CSG geometries for neutronics benchmarking*. https://github.com/fusion-energy/model_benchmark_zoo

Shimwell, J., Billingsley, J., Delaporte-Mathurin, R., Morbey, D., Bluteau, M., Shriwise, P., & Davis, A. (2021). The Paramak: Automated parametric geometry construction for fusion reactor designs. *F1000Research*, *10*, 27. https://doi.org/10.12688/f1000research.28224.1

Shimwell, J., Labrie-Cleary, L., & Gray, A. (2022). *cad-to-h5m: Converts CAD files to a DAGMC h5m file using Cubit* (Version 0.3.0). https://github.com/fusion-energy/cad_to_h5m

Shimwell, J., & Shriwise, P. (2022). *STL to h5m: Convert non-overlapping STL files into a DAGMC h5m file complete with material tags* (Version 0.2.3). https://github.com/fusion-energy/stl_to_h5m

Shriwise, P. C., Wilson, P. P. H., Davis, A., & Romano, P. K. (2022). Hardware-accelerated ray tracing of CAD-based geometry for Monte Carlo radiation transport. *Computing in Science & Engineering*, *24*(2), 52–61. https://doi.org/10.1109/MCSE.2022.3154656

Siemens Digital Industries Software. (2026). *Parasolid*. https://www.siemens.com/en-us/products/plm-components/parasolid/

Smith, B. M., Tautges, T. J., & Wilson, P. P. H. (2010). Sealing faceted surfaces to achieve watertight CAD models. In S. Shontz (Ed.), *Proceedings of the 19th international meshing roundtable* (pp. 177–194). Springer. https://doi.org/10.1007/978-3-642-15414-0_11

Swanson, C. P. S., Kumar, S. T. A., Dudt, D. W., Flom, E. R., Kalb, W. B., Kruger, T. G., Martin, M. F., Olatunji, J. R., Pasmann, S., Tang, L. Z., Linden, J. von der, Wasserman, J., Avida, M., Basurto, A. S., Dickerson, M., Boer, N. de, Donovan, M. J., Doudna Cate, A. H., Fort, D., … Gates, D. A. (2026). Overview of the Helios design: A practical planar coil stellarator fusion power plant. *Fusion Engineering and Design*, *232*, 115999. https://doi.org/10.1016/j.fusengdes.2026.115999

Tautges, T. J., Meyers, R., Merkley, K., Stimpson, C., & Ernst, C. (2004). *MOAB: A mesh-oriented database* (No. SAND2004-1592). Sandia National Laboratories. https://doi.org/10.2172/970174

The UW CNERG Team. (2010). *Using DAG-MCNP5*. https://web.archive.org/web/20100610210935/http://cnerg.engr.wisc.edu/cgi-bin/index.cgi/wiki?p=UsingDagmc

The UW CNERG Team. (2014). *The University of Wisconsin unified workflow*. https://web.archive.org/web/20141125063910/http://svalinn.github.io/DAGMC/usersguide/uw2.html

The UW CNERG Team. (2022a). *DAGMC: Direct accelerated geometry Monte Carlo toolkit documentation*. https://svalinn.github.io/DAGMC/

DRAFT

The UW CNERG Team. (2022b). *Svalinn Cubit plugin: Plugins and command extensions for Coreform Cubit* (Version 0.2.4). https://github.com/svalinn/Cubit-plugin

Wilson, P. P. H., Tautges, T. J., Kraftcheck, J. A., Smith, B. M., & Henderson, D. L. (2010). Acceleration techniques for the direct use of CAD-based geometry in fusion neutronics analysis. *Fusion Engineering and Design*, *85*(10–12), 1759–1765. https://doi.org/10.1016/j.fusengdes.2010.05.030

DRAFT